Nahjay Battieste

954-494-7138 nahjaybattieste@gmail.com www.linkedin.com/in/nahjay-battieste-a84655224 https://github.com/Nahjay
Portfolio | https://nahjay.github.io/

Education

University of Florida **Aug. 2024 – Expected May 2027**
Bachelor of Science in Computer Engineering | GPA: 3.93 *Gainesville, Florida*

Broward College **Aug. 2022 – May 2024**
Associates of Arts | GPA: 3.95 *Coconut Creek, Florida*

Technical Skills

Languages: C, C++, Python, VHDL, SystemVerilog, Rust, Assembly, Bash, PowerShell, MATLAB
Hardware / FPGA: Quartus, Vivado
Systems & Tools: Perforce, Docker, GNU Make, Linux, Git, Claude Code, OpenAI Codex

Experience

Undergraduate Teaching Assistant — University of Florida **January 2026 – Current**
Digital Design (EEL4712C) *Gainesville, Florida*

- Assisted in instruction of digital logic and hardware design concepts, including FSMs, datapath/control separation, pipelining, and timing analysis using SystemVerilog, while leading lab support sessions and debugging student designs through RTL simulation, synthesis, and FPGA deployment.
- Reviewed and graded assignments focused on register-transfer level design and hardware verification.

SmartSystems Lab — University of Florida **October 2024 – Current**
Undergraduate Researcher *Gainesville, Florida*

- Designing an FPGA-based CNN accelerator for AI inference, including a custom frontend architecture for convolution dataflow, line-buffered sliding-window generation, and memory structures for feature-map reuse across 3-channel image workloads.
- Integrating accelerator datapaths with compute and system interfaces, including systolic-array-oriented modules, AXI4/AXI4-Stream communication, configuration registers, and clock-domain-crossing logic for SoC-style deployment, while improving timing closure and throughput through RTL refinement and pipelining.

NVIDIA **May 2026 – August 2026**
GPU Firmware Engineer Intern, Chips Team *Santa Clara, California*

- Rearchitected the settings (register-configuration) model for the GPU hardware-initialization boot microcode across NVIDIA's next-generation Rubin-architecture client GPUs (desktop, workstation, and notebook SKUs), consolidating structure definitions out of 40+ per-module C header files into a single schema-generated header derived from a unified configuration schema.
- Redesigned the microcode's build and linking process so configuration data could be packaged directly into the microcode's own data memory alongside its execution sequence, letting settings and code move and deploy together as one containerized unit for the first time.
- Built a build-time "burn" capability that collects finalized per-chip configuration values and writes them into the reserved data-memory region of the packaged microcode image, producing a fully self-contained, chip-specific boot image.

NVIDIA **May 2025 – August 2025**
GPU Firmware Engineer Intern, Infrastructure Team *Santa Clara, California*

- Developed a remote testing pipeline to validate firmware on target hardware, automating GPU ROM data extraction to configure test environments and execute Pytest suites.
- Created an automated regression hunting tool that performs a binary search across Perforce changelists to pinpoint commits responsible for build failures or performance degradation in the firmware.
- Enhanced a containerized development environment for firmware builds and compilation, building upon a previous project to further streamline the build process to increase productivity and reduce errors for engineers.

NVIDIA **May 2024 – August 2024**
GPU Firmware Engineer Intern, Infrastructure Team *Santa Clara, California*

- Developed a wizard and series of scripts using Python, Batch, Bash, PowerShell, and Docker to create custom containerized environments used to build and develop firmware packages and microcode components, significantly reducing ramp-up time for new team members and development time for existing engineers.
- Optimized boot instantiation sequence in C for next-generation GPU architecture, removing obsolete code and integrating new architectural features to ensure compatibility and performance.
- Identified and resolved MASM build issues for core architectures using assembly language, updating assembler tooling, correcting directives for ROM data table structures, and improving assembly file performance.

Cell Antenna Corporation **May 2023 – Apr 2024**
Electrical Engineer Intern *Coral Springs, Florida*

- Engineered a full-stack direction-finding system using FastAPI and Rust for the backend, and a JavaScript web interface, to visualize real-time signal data from an antenna array.
- Developed a GUI (Python/DearPyGui) and a Rust-based asynchronous API (Actix-Web) to streamline control of a Yocto Linux-based SDR and a GPS device, improving user efficiency.
- Wrote and optimized time-critical firmware in Assembly for an Atmel microcontroller synchronized to a GPS PPS signal and developed a Rust program to process raw IQ data from an FPGA-based SDR.

Projects

UF/Apple-Sponsored Hardware Design Contest, BNN FPGA Accelerator | *SystemVerilog*

- Optimized a parameterized FPGA BNN accelerator to 670.69 MHz fMax on Xilinx UltraScale via pipelining, retiming, and LUT reduction strategies, targeting the SFC 784→256→256→10 topology for MNIST digit recognition.
- Designed full RTL in SystemVerilog for a pipelined binary neural network classifier with AXI4-Stream config, input, and output interfaces; implemented a custom configuration manager, neuron processors (PN=8, PW=8), and memory hierarchy across 3 hidden/output layers.
- Developed a parameterized SystemVerilog testbench with functional coverage, assertions, and a behavioral reference model; achieved 100% verification pass rate across 50 test cases with measured average throughput of 21,545 outputs/sec.

Jetson Nano Custom Kernel Modules | *C, CUDA*

- Integrated a Linux kernel module with GPU-accelerated CUDA kernels into an image-processing application on the NVIDIA Jetson Nano.
- Designed and optimized CUDA kernels to exploit GPU parallelism for accelerated execution compared with CPU-only processing.
- Managed kernel–user space coordination and data movement to support efficient CPU–GPU interaction in an embedded Linux environment.